Лабораторная работа № 2
Реализация стохастических методов оптимизации для обучения нейронных сетей

Цель работы реализовать и исследовать стохастические методы оптимизации, применяемые при обучении глубоких нейронных сетей, а также развить понимание их математических свойств, областей применимости и влияния на процесс обучения.

Задачи:

1. Изучить математическую постановку задачи обучения нейронной сети как задачи минимизации невыпуклой функции потерь.
2. Реализовать базовые стохастические оптимизаторы:
   – SGD;
   – SGD с momentum;
   – Adam;
   – AdamW.
3. Исследовать траектории сходимости оптимизаторов на тестовых невыпуклых функциях двух переменных.
4. Применить реализованные оптимизаторы для обучения нейронной сети на реальной задаче классификации или регрессии.
5. Проанализировать влияние размера мини-пакета и скорости обучения на скорость сходимости, устойчивость обучения и качество обобщения.

1. Теоретическая справка

В работа рассматриваются вопросы реализации математического аппарата глубокого обучения. В работе связываются следующие темы курса:

– автоматическое дифференцирование;
– невыпуклая оптимизация;
– теория обобщения;
– полносвязные нейронные сети;
– стохастические методы обучения.

Данная работа важна потому, что позволяет перейти от формального вызова оптимизаторов из библиотеки к пониманию того:

– как именно обновляются параметры модели;
– почему одни методы сходятся быстрее, но обобщают хуже;
– как шум мини-пакетного градиента влияет на траекторию обучения;
– чем регуляризация через L_2 отличается от weight decay в адаптивных методах.

1.1. Постановка задачи обучения нейронной сети

Пусть задана выборка пар объект–ответ:

$$\{(x_i, y_i)\}_{i=1}^{N}, \tag{2.1}$$

где x_i — входной объект, y_i — целевое значение, N — объём выборки.

Нейронная сеть задаёт параметрическую модель:

$$f_\theta(x), \tag{2.2}$$

где $\theta \in \mathbb{R}^d$ — вектор обучаемых параметров.

Обучение сети сводится к минимизации эмпирического риска:

$$L(\theta) = \frac{1}{N}\sum_{i=1}^{N} \ell\left(f_\theta(x_i), y_i\right), \tag{2.3}$$

где $L(\theta)$ — функция потерь на всей выборке;
$\ell(\cdot,\cdot)$ — функция потерь для одного объекта;
θ — параметры модели.

В глубоком обучении функция $L(\theta)$, как правило, является:
- невыпуклой;
- многомодальной;
- содержащей седловые точки;
- трудно минимизируемой глобально.

> **Замечание.**
> В отличие от классической выпуклой оптимизации, в глубоком обучении часто целью является не нахождение глобального минимума, а нахождение достаточно хорошего локального минимума с приемлемой обобщающей способностью.

1.2. Мини-пакетный градиент и стохастическая аппроксимация

Полный градиент по всей выборке обычно вычисляется слишком долго. Поэтому используется мини-пакетная оценка градиента.

Пусть $\mathcal{B}$ — мини-пакет размера B. Тогда оценка градиента:

$$g_t = \frac{1}{B}\sum_{i\in\mathcal{B}} \nabla_\theta \ell(f_\theta(x_i), y_i), \tag{2.4}$$

где g_t — оценка градиента на шаге t;
B — размер мини-пакета;
∇_θ — градиент по параметрам модели.

Обновление параметров в простейшем случае имеет вид:

$$\theta_{t+1} = \theta_t - \eta g_t, \tag{2.5}$$

где $\eta > 0$ — скорость обучения;
θ_t — параметры на шаге t.

> **Замечание.**
> Мини-пакетный градиент является несмещённой оценкой полного градиента, но обладает дисперсией, которая уменьшается с ростом B.

1.3. SGD с инерцией

Классический SGD может колебаться в узких оврагах и медленно двигаться по направлениям с малым градиентом. Для ускорения сходимости используется инерция.

Один из распространённых вариантов обновления:

$$v_{t+1} = \gamma v_t + g_t, \tag{2.6}$$
$$\theta_{t+1} = \theta_t - \eta v_{t+1}, \tag{2.7}$$

где v_t — вектор скорости;
$\gamma \in [0,1)$ — коэффициент инерции, обычно 0.9;
g_t — текущий градиент.

Идея метода состоит в накоплении градиентов вдоль устойчивых направлений и сглаживании осцилляций.

> **Замечание.**
> В литературе и в PyTorch встречаются немного разные формы записи momentum. В некоторых источниках коэффициент обучения включается в обновление скорости, а в других — применяется на шаге обновления параметров. При реализации и сравнении с библиотечными оптимизаторами это необходимо учитывать отдельно**.**

1.4. Adam: адаптивная оценка моментов

Алгоритм Adam адаптирует шаг обучения для каждого параметра отдельно, используя экспоненциальные скользящие средние первого и второго моментов градиента.

Оценка первого момента:

$$m_{t+1} = \beta_1 m_t + (1 - \beta_1) g_t, \tag{2.8}$$

где m_t — оценка среднего градиента;
β_1 — коэффициент затухания первого момента, обычно 0.9.

Оценка второго момента:

$$v_{t+1} = \beta_2 v_t + (1 - \beta_2) g_t^2, \tag{2.9}$$

где v_t — оценка немасштабированного второго момента;
β_2 — коэффициент затухания второго момента, обычно 0.999;
g_t^2 — поэлементный квадрат градиента.

Поскольку начальные значения $m_0 = 0$, $v_0 = 0$, оценки на первых шагах смещены к нулю. Для коррекции используется поправка на смещение:

$$\widehat{m}_{t+1} = \frac{m_{t+1}}{1-\beta_1^{t+1}}, \tag{2.10}$$

$$\hat{v}_{t+1} = \frac{v_{t+1}}{1-\beta_2^{t+1}}. \tag{2.11}$$

Обновление параметров:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_{t+1}}+\varepsilon} \widehat{m}_{t+1}, \tag{2.12}$$

где η — скорость обучения;
$\varepsilon > 0$ — малая константа для предотвращения деления на ноль, обычно 10^{-8};
$\widehat{m}_{t+1}$ — скорректированная оценка первого момента;
$\hat{v}_{t+1}$ — скорректированная оценка второго момента.

> **Упражнение.**
> Покажите, что если g_t в начале обучения можно считать приблизительно постоянным, то без поправки на смещение эффективный шаг в Adam на первых итерациях будет существенно меньше номинального η.

1.5. AdamW и отличие weight decay от L_2-регуляризации

В классической постановке L_2-регуляризация добавляется к функции потерь:

$$L_{\text{reg}}(\theta) = L(\theta) + \frac{\lambda}{2} \| \theta \|^2, \tag{2.13}$$

что приводит к добавлению слагаемого $\lambda\theta$ к градиенту.

В адаптивных оптимизаторах такая форма регуляризации взаимодействует с масштабированием шага по параметрам и может работать хуже. В AdamW применяется «decoupled weight decay» (раздельное затухание весов), то есть весовые коэффициенты «weight decay» отделяются от адаптивного градиентного шага:

$$\theta_{t+1} = \theta_t - \eta \left(\frac{\widehat{m}_{t+1}}{\sqrt{\hat{v}_{t+1}}+\varepsilon} + \lambda \theta_t \right), \tag{2.14}$$

где λ — коэффициент «weight decay»;
$\lambda\theta_t$ — отдельное штрафное слагаемое, не проходящее через адаптивные оценки моментов.

> **Типичная ошибка.**
> Студенты часто считают, что L_2-регуляризация и weight decay — это всегда одно и то же. В SGD при определённых условиях они эквивалентны, но в Adam и AdamW это разные механизмы, и именно «decoupled weight decay» часто даёт лучшее качество обобщения.

1.6. Краткий исторический контекст

- стохастические методы оптимизации восходят к работам середины XX века;
- идея инерции в оптимизации связана с методами тяжелой шаровой частицы и работами Б. Т. Поляка;
- метод Adam был предложен в работе Д. Кингмы и Дж. Ба в 2014 году;

– метод AdamW был предложен И. Лощиловым и Ф. Хуттером в 2017 году как модификация, улучшающая обобщение за счёт корректного «весового затухания».

2. Ход выполнения работы

2.1. Рекомендуемое программное обеспечение

– Python 3.10 или новее;
– PyTorch;
– NumPy;
– Matplotlib;
– при необходимости — TensorFlow/Keras для альтернативной реализации.

2.2. Данные

Для основной части работы используются встроенные наборы данных, например: MNIST; FashionMNIST; KMNIST; CIFAR-10; CIFAR-100; California Housing для задачи регрессии.

Выбор конкретного набора данных определяется индивидуальным вариантом.

2.3. Пошаговое задание с контрольными точками

Шаг 0. Фиксация случайности и подготовка окружения

Для обеспечения воспроизводимости эксперимента зафиксируйте случайные генераторы.

Листинг 2.1. Инициализация начального состояния среды программирования

```
import os
import random

import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
import matplotlib.pyplot as plt
import math
def set_seed(seed: int = 42) -> None:
    """
    Фиксирует случайные генераторы для воспроизводимости эксперимента.
    """
    os.environ["PYTHONHASHSEED"] = str(seed)
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False
set_seed(42)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Используемое устройство:", device)
```

> **Замечание.**
> Фиксация случайности особенно важна при сравнении оптимизаторов, иначе различия в результатах могут быть вызваны разным разбиением на пакеты или разной инициализацией весов.

Шаг 1. Реализация пользовательских оптимизаторов в PyTorch

Ниже приведены учебные примеры реализации четырёх оптимизаторов.

Листинг 2.2. Реализация метода SGD

```
from torch.optim import Optimizer

class CustomSGD(Optimizer):
    """
    Учебная реализация классического стохастического градиентного спуска.
    """
    def __init__(self, params, lr: float = 1e-2, weight_decay: float = 0.0):
        defaults = dict(lr=lr, weight_decay=weight_decay)
        super().__init__(params, defaults)

    @torch.no_grad()
    def step(self, closure=None):
        """
        Выполняет один шаг обновления параметров.
        """
        loss = None
        if closure is not None:
            with torch.enable_grad():
                loss = closure()

        for group in self.param_groups:
            lr = group["lr"]
            weight_decay = group["weight_decay"]
            for p in group["params"]:
                if p.grad is None:
                    continue
                grad = p.grad
                # Добавляем L2-составляющую к градиенту, если она задана
                if weight_decay != 0:
                    grad = grad.add(p, alpha=weight_decay)
                # Обновление параметров
                p.add_(grad, alpha=-lr)
        return loss
```

Листинг 2.3. Реализация метода SGD с momentum

```
class CustomMomentum(Optimizer):
    """
    Учебная реализация SGD с momentum.
    Используется схема:
        v_t = gamma * v_{t-1} + g_t
        theta_t = theta_{t-1} - lr * v_t
    """
    def __init__(self, params, lr: float = 1e-2, momentum: float = 0.9, weight_
decay: float = 0.0):
        defaults = dict(lr=lr, momentum=momentum, weight_decay=weight_decay)
        super().__init__(params, defaults)
    @torch.no_grad()
    def step(self, closure=None):
        loss = None
        if closure is not None:
            with torch.enable_grad():
```

```
                loss = closure()
        for group in self.param_groups:
            lr = group["lr"]
            momentum = group["momentum"]
            weight_decay = group["weight_decay"]
            for p in group["params"]:
                if p.grad is None:
                    continue
                grad = p.grad
                # L2-регуляризация через градиент
                if weight_decay != 0:
                    grad = grad.add(p, alpha=weight_decay)
                state = self.state[p]
                # Инициализация буфера инерции при первом вызове
                if "momentum_buffer" not in state:
                    state["momentum_buffer"] = torch.zeros_like(p)

                buf = state["momentum_buffer"]

                # Накопление скорости
                buf.mul_(momentum).add_(grad)

                # Обновление параметров
                p.add_(buf, alpha=-lr)
        return loss
```

Листинг 2.4. Реализация методов Adam и AdamW

```
class CustomAdamW(Optimizer):
    """
    Учебная реализация AdamW.
    При weight_decay=0 фактически превращается в классический Adam.
    """
    def __init__(
        self,
        params,
        lr: float = 1e-3,
        betas=(0.9, 0.999),
        eps: float = 1e-8,
        weight_decay: float = 0.0
    ):
        defaults = dict(lr=lr, betas=betas, eps=eps, weight_decay=weight_decay)
        super().__init__(params, defaults)
    @torch.no_grad()
    def step(self, closure=None):
        loss = None
        if closure is not None:
            with torch.enable_grad():
                loss = closure()

        for group in self.param_groups:
            lr = group["lr"]
            beta1, beta2 = group["betas"]
            eps = group["eps"]
            weight_decay = group["weight_decay"]
            for p in group["params"]:
```

```
                if p.grad is None:
                    continue
                grad = p.grad
                state = self.state[p]
                # Инициализация состояния оптимизатора
                if len(state) == 0:
                    state["step"] = 0
                    state["exp_avg"] = torch.zeros_like(p)
                    state["exp_avg_sq"] = torch.zeros_like(p)
                exp_avg, exp_avg_sq = state["exp_avg"], state["exp_avg_sq"]
                state["step"] += 1
                # Decoupled weight decay: сначала уменьшаем сами веса
                if weight_decay != 0:
                    p.mul_(1.0 - lr * weight_decay)
                # Обновление оценок первого и второго моментов
                exp_avg.mul_(beta1).add_(grad, alpha=1.0 - beta1)
                exp_avg_sq.mul_(beta2).addcmul_(grad, grad, value=1.0 - beta2)
                step = state["step"]
                # Поправка на смещение
                bias_correction1 = 1.0 - beta1 ** step
                bias_correction2 = 1.0 - beta2 ** step
                step_size = lr / bias_correction1
                denom = (exp_avg_sq.sqrt() / math.sqrt(bias_correction2)).add_(
eps)
                # Адаптивное обновление параметров
                p.addcdiv_(exp_avg, denom, value=-step_size)
        return loss
```

Шаг 2. Исследование оптимизаторов на тестовых 2D-функциях

Цель данного этапа — визуально сравнить траектории оптимизаторов на невыпуклой поверхности. Рассмотрим решение задачи на примере функции Розенброка

Функция Розенброка задаётся формулой:

$$f(x,y) = (1-x)^2 + 100(y-x^2)^2. \tag{2.14}$$

Она имеет узкий изогнутый овраг и часто используется для проверки алгоритмов оптимизации.

Листинг 2.5. Универсальная функция запуска оптимизатора на 2D-функции

```
def run_optimizer_on_2d(
    optimizer_class,
    start_point,
    lr: float,
    n_steps: int = 300,
    **optimizer_kwargs
):
    """
    Запускает пользовательский оптимизатор на 2D-функции Розенброка.
    Возвращает:
        trajectory - массив точек траектории;
        final_loss - значение функции в конце оптимизации.
    """
    point = torch.tensor(start_point, dtype=torch.float32, requires_grad=True)
    optimizer = optimizer_class([point], lr=lr, **optimizer_kwargs)
    trajectory = [point.detach().clone().numpy()]
    for _ in range(n_steps):
        optimizer.zero_grad()
```

```
        x, y = point[0], point[1]
        # Функция Розенброка
        loss = (1.0 - x) ** 2 + 100.0 * (y - x ** 2) ** 2
        loss.backward()
        optimizer.step()
        trajectory.append(point.detach().clone().numpy())
    trajectory = np.array(trajectory)
    final_loss = loss.item()
    return trajectory, final_loss
```

Листинг 2.6. Запуск четырёх оптимизаторов

```
start = [-1.5, 1.5]

trajectories = {}
final_losses = {}

trajectories["SGD"], final_losses["SGD"] = run_optimizer_on_2d(
    CustomSGD,
    start_point=start,
    lr=1e-3,
    n_steps=300
)

trajectories["Momentum"], final_losses["Momentum"] = run_optimizer_on_2d(
    CustomMomentum,
    start_point=start,
    lr=1e-3,
    momentum=0.9,
    n_steps=300
)

trajectories["Adam"], final_losses["Adam"] = run_optimizer_on_2d(
    CustomAdamW,
    start_point=start,
    lr=1e-2,
    weight_decay=0.0,
    n_steps=300
)

trajectories["AdamW"], final_losses["AdamW"] = run_optimizer_on_2d(
    CustomAdamW,
    start_point=start,
    lr=1e-2,
    weight_decay=0.01,
    n_steps=300
)

for name, loss_value in final_losses.items():
    print(f"{name}: конечное значение функции = {loss_value:.6f}")
```

Листинг 2.7. Построение траекторий

```
def plot_trajectories(trajectories):
    """
    Строит линии уровня функции Розенброка и траектории оптимизаторов.
    """
```

```
    x = np.linspace(-2.0, 2.0, 300)
    y = np.linspace(-1.0, 3.0, 300)
    X, Y = np.meshgrid(x, y)
    Z = (1 - X) ** 2 + 100 * (Y - X ** 2) ** 2
    plt.figure(figsize=(8, 6))
    plt.contour(X, Y, Z, levels=50, cmap="viridis", alpha=0.6)

    for name, traj in trajectories.items():
        plt.plot(traj[:, 0], traj[:, 1], label=name, linewidth=2)
        plt.scatter(traj[-1, 0], traj[-1, 1], s=30)
    plt.xlabel("x")
    plt.ylabel("y")
    plt.title("Траектории оптимизаторов на функции Розенброка")
    plt.legend()
    plt.grid(True)
    plt.show()
plot_trajectories(trajectories)
```

По итогам данного этапа необходимо представить:

1. Работающий код всех четырёх пользовательских оптимизаторов.
2. График траекторий на тестовой функции.
3. Таблицу конечных значений функции для каждого оптимизатора.
4. Краткий вывод о характере сходимости каждого метода.

Шаг 3. Обучение нейронной сети с пользовательскими оптимизаторами

Далее необходимо применить реализованные оптимизаторы к обучению нейронной сети. Ниже приведён пример для задачи классификации рукописных цифр. В индивидуальном варианте набор данных и архитектура могут отличаться.

Листинг 2.8. Загрузка данных

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])
train_dataset = datasets.MNIST(
    root="./data",
    train=True,
    download=True,
    transform=transform
)
test_dataset = datasets.MNIST(
    root="./data",
    train=False,
    download=True,
    transform=transform
)
train_loader = DataLoader(train_dataset, batch_size=128, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=256, shuffle=False)
```

Листинг 2.9. Определение модели

```
class SimpleMLP(nn.Module):
    """
    Простая полносвязная сеть для классификации изображений 28x28.
    """
    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(28 * 28, 128)
        self.fc2 = nn.Linear(128, 64)
        self.fc3 = nn.Linear(64, 10)
    def forward(self, x):
        x = self.flatten(x)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x
```

Листинг 2.10. Функция обучения одной эпохи

```
def train_one_epoch(model, loader, optimizer, criterion, device):
    """
    Обучает модель одну эпоху.
    """
    model.train()
    total_loss = 0.0
    total_correct = 0
    total_samples = 0

    for x_batch, y_batch in loader:
        x_batch = x_batch.to(device)
        y_batch = y_batch.to(device)

        optimizer.zero_grad()

        logits = model(x_batch)
        loss = criterion(logits, y_batch)

        loss.backward()
        optimizer.step()

        total_loss += loss.item() * x_batch.size(0)
        total_correct += (logits.argmax(dim=1) == y_batch).sum().item()
        total_samples += x_batch.size(0)

    avg_loss = total_loss / total_samples
    accuracy = total_correct / total_samples

    return avg_loss, accuracy
```

Листинг 2.11. Функция оценки на тестовой выборке

```
@torch.no_grad()
def evaluate(model, loader, criterion, device):
    """
    Вычисляет функцию потерь и точность на заданном датасете.
    """
    model.eval()
    total_loss = 0.0
    total_correct = 0
    total_samples = 0
    for x_batch, y_batch in loader:
        x_batch = x_batch.to(device)
        y_batch = y_batch.to(device)
        logits = model(x_batch)
        loss = criterion(logits, y_batch)
        total_loss += loss.item() * x_batch.size(0)
        total_correct += (logits.argmax(dim=1) == y_batch).sum().item()
        total_samples += x_batch.size(0)
    avg_loss = total_loss / total_samples
    accuracy = total_correct / total_samples

    return avg_loss, accuracy
```

Листинг 2.12. Полный цикл обучения

```
def run_experiment(
    model_class,
    optimizer_class,
    lr: float,
    epochs: int = 5,
    batch_size: int = 128,
    optimizer_kwargs=None,
    device=device
):
    """
    Полный цикл обучения модели с выбранным оптимизатором.
    """
    torch.manual_seed(42)
    model = model_class().to(device)
    criterion = nn.CrossEntropyLoss()
    train_loader_local = DataLoader(
        train_dataset,
        batch_size=batch_size,
        shuffle=True
    )
    test_loader_local = DataLoader(
        test_dataset,
        batch_size=256,
        shuffle=False
    )
    if optimizer_kwargs is None:
        optimizer_kwargs = {}
    optimizer = optimizer_class(
        model.parameters(),
        lr=lr,
        **optimizer_kwargs
```

```
    )

    history = {
        "train_loss": [],
        "train_acc": [],
        "test_loss": [],
        "test_acc": []
    }
    for epoch in range(1, epochs + 1):
        train_loss, train_acc = train_one_epoch(
            model,
            train_loader_local,
            optimizer,
            criterion,
            device
        )
        test_loss, test_acc = evaluate(
            model,
            test_loader_local,
            criterion,
            device
        )
        history["train_loss"].append(train_loss)
        history["train_acc"].append(train_acc)
        history["test_loss"].append(test_loss)
        history["test_acc"].append(test_acc)
        print(
            f"Эпоха {epoch:02d}: "
            f"train_loss={train_loss:.4f}, train_acc={train_acc:.4f}, "
            f"test_loss={test_loss:.4f}, test_acc={test_acc:.4f}"
        )
    return history
```

Листинг 2.13. Запуск экспериментов с разными оптимизаторами

```
results = {}

results["SGD"] = run_experiment(
    SimpleMLP,
    CustomSGD,
    lr=0.05,
    epochs=5,
    batch_size=128
)
results["Momentum"] = run_experiment(
    SimpleMLP,
    CustomMomentum,
    lr=0.05,
    epochs=5,
    batch_size=128,
    optimizer_kwargs={"momentum": 0.9}
)
results["Adam"] = run_experiment(
    SimpleMLP,
    CustomAdamW,
    lr=0.001,
```

```
    epochs=5,
    batch_size=128,
    optimizer_kwargs={"weight_decay": 0.0}
)
results["AdamW"] = run_experiment(
    SimpleMLP,
    CustomAdamW,
    lr=0.001,
    epochs=5,
    batch_size=128,
    optimizer_kwargs={"weight_decay": 0.01}
)
```

Листинг 2.14. Построение графиков обучения

```
def plot_metric(results, metric_name, title):
    """
    Строит график метрики по эпохам для нескольких оптимизаторов.
    """
    plt.figure(figsize=(8, 5))
    for name, history in results.items():
        plt.plot(history[metric_name], label=name, linewidth=2)
    plt.xlabel("Эпоха")
    plt.ylabel(metric_name)
    plt.title(title)
    plt.grid(True)
    plt.legend()
    plt.show()
plot_metric(results, "train_loss", "Обучающая функция потерь")
plot_metric(results, "test_acc", "Точность на тестовой выборке")
```

По итогам этапа необходимо представить:

1. Графики обучающей и тестовой функции потерь.
2. Графики точности на обучающей и тестовой выборках.
3. Итоговую таблицу:

– оптимизатор;
– обучаемость;
– тестовая точность;
– тестовая ошибка;
– время обучения.

4. Вывод о том, какой оптимизатор оказался наиболее устойчивым и быстрым.

Шаг 4. Анализ влияния гиперпараметров

На данном этапе исследуется влияние размера мини-пакета и скорости обучения.

Листинг 2.15. Рекомендуемая сетка эксперимента

```
batch_sizes = [16, 64, 256, 1024]
learning_rates = [1e-1, 1e-2, 1e-3, 1e-4]
```

> **Важно!**
> Слишком большая скорость обучения может привести к расходимости, а слишком маленькая — к крайне медленной сходимости. Размер пакета влияет на дисперсию градиентной оценки и на характер траектории в пространстве параметров.

Листинг 2.16. Пример упрощённого перебора

```
def run_grid_experiment(optimizer_class, optimizer_name):
    """
    Упрощённый перебор по скорости обучения и размеру пакета.
    """
    grid_results = []

    for batch_size in batch_sizes:
        for lr in learning_rates:
            print(f"\nЗапуск: {optimizer_name}, batch_size={batch_size}, lr={lr
}")
            try:
                history = run_experiment(
                    SimpleMLP,
                    optimizer_class,
                    lr=lr,
                    epochs=2,
                    batch_size=batch_size,
                    device=device
                )
                final_test_acc = history["test_acc"][-1]
                final_test_loss = history["test_loss"][-1]
                grid_results.append({
                    "optimizer": optimizer_name,
                    "batch_size": batch_size,
                    "lr": lr,
                    "final_test_acc": final_test_acc,
                    "final_test_loss": final_test_loss
                })
            except Exception as e:
                print("Ошибка при обучении:", e)
                grid_results.append({
                    "optimizer": optimizer_name,
                    "batch_size": batch_size,
                    "lr": lr,
                    "final_test_acc": None,
                    "final_test_loss": None
                })
    return grid_results
```

Необходимо представить:

1. Результаты перебора гиперпараметров в табличном виде.
2. Тепловую карту или график зависимости качества от `(batch_size, learning_rate)`.
3. Выводы:

– как размер пакета влияет на устойчивость обучения;
– какие диапазоны скоростей обучения оказались рабочими;
– наблюдалась ли деградация обобщения при слишком малом пакете.

2.4. Альтернативный фрагмент на TensorFlow / Keras

Ниже приведён альтернативный пример ручной реализации одного шага AdamW для задачи на 2D-функции. Он предназначен для сопоставления логики алгоритма между PyTorch и TensorFlow.

Листинг 2.17. Альтернативный фрагмент на TensorFlow / Keras

```
import tensorflow as tf


# Начальная точка
point = tf.Variable([-1.5, 1.5], dtype=tf.float32)


# Гиперпараметры
lr = 0.01
beta_1 = 0.9
beta_2 = 0.999
eps = 1e-8
weight_decay = 0.01
n_steps = 300

# Оценки моментов
m = tf.Variable(tf.zeros_like(point), dtype=tf.float32)
v = tf.Variable(tf.zeros_like(point), dtype=tf.float32)

for step in range(1, n_steps + 1):
    with tf.GradientTape() as tape:
        x, y = point[0], point[1]
        loss = (1.0 - x) ** 2 + 100.0 * (y - x ** 2) ** 2

    grad = tape.gradient(loss, point)


    # Decoupled weight decay
    point.assign_sub(lr * weight_decay * point)

    # Обновление моментов
    m.assign(beta_1 * m + (1.0 - beta_1) * grad)
    v.assign(beta_2 * v + (1.0 - beta_2) * grad * grad)

    # Поправка на смещение
    m_hat = m / (1.0 - beta_1 ** step)
    v_hat = v / (1.0 - beta_2 ** step)

    # Адаптивный шаг
    point.assign_sub(lr * m_hat / (tf.sqrt(v_hat) + eps))

print("Итоговая точка:", point.numpy())
print("Итоговое значение функции:", loss.numpy())
```

Для обучения нейронной сети в Keras можно использовать встроенные оптимизаторы.

Листинг 2.18. Создание модели нейронной сети на TensorFlow / Keras

```
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dense(64, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax")
])

model.compile(
```

```
    optimizer=tf.keras.optimizers.SGD(learning_rate=0.05, momentum=0.9),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

> **Замечание.**
> В рамках данной лабораторной основной акцент делается на самостоятельной реализации алгоритмов. Поэтому PyTorch является основным средством выполнения работы, а TensorFlow/Keras используется как альтернатива для сравнения.

3. Критерии оценивания

Таблица 2.1. Критерии оценивания

Критерий	**Баллы**	**Описание**
Реализация оптимизаторов	1	Корректно реализованы SGD, Momentum, Adam/AdamW, код работоспособен
Исследование на 2D-функции	2	Построены траектории, проведено сравнение, графики аккуратны
Обучение нейронной сети	2	Модель обучена, графики и метрики представлены для всех оптимизаторов
Анализ гиперпараметров	2	Проведён эксперимент по batch_size и lr, сделаны содержательные выводы
Защита работы	3	Студент отвечает на вопросы, понимает математическую суть методов
Итого	**10**	

Таблица 2.2. Шкала перевода баллы в оценки

Баллы	Оценка
9–10	Отлично
7–8	Хорошо
5–6	Удовлетворительно
< 5	Неудовлетворительно (доработка)

4. Чек-лист готовности к защите

После выполнения лабораторной работы студент должен уметь:

1) формулировать задачу обучения нейронной сети как задачу невыпуклой оптимизации;
2) объяснять различие между полным, стохастическим и мини-пакетным градиентом;
3) реализовывать вручную базовые оптимизаторы:
   – SGD;
   – SGD с momentum;
   – Adam;
   – AdamW;
4) объяснять назначение поправки на смещение в Adam;
5) различать L_2-регуляризацию и decoupled weight decay;
6) анализировать траектории оптимизации на тестовых функциях;
7) оценивать влияние скорости обучения и размера пакета на устойчивость обучения;
8) корректно сравнивать оптимизаторы по графикам обучения и тестовым метрикам;

9) воспроизводимо проводить нейросетевые эксперименты с фиксацией случайности;
10) интерпретировать полученные результаты с позиции математика, а не только пользователя библиотек.

5. Варианты индивидуальных заданий

Ниже приведены 10 вариантов. Каждый вариант задаёт:

- тестовую 2D-функцию для визуализации траекторий;
- стартовую точку;
- набор данных и тип модели для основной части;
- дополнительные акценты эксперимента.

5.1. Перечень тестовых функций

1. **Розенброк**

$$f(x,y) = (1-x)^2 + 100(y-x^2)^2$$

Старт: $(-1.5, 1.5)$

2. **Химмельблау**

$$f(x,y) = (x^2 + y - 11)^2 + (x + y^2 - 7)^2$$

Старт: $(0.5, -0.5)$

3. **Била**

$$f(x,y) = (1.5 - x + xy)^2 + (2.25 - x + xy^2)^2 + (2.625 - x + xy^3)^2$$

Старт: $(0,0)$

4. **Растригин**

$$f(x,y) = 20 + x^2 + y^2 - 10\big(\cos(2\pi x) + \cos(2\pi y)\big)$$

Старт: $(2.5, 2.5)$

5. **Бута**

$$f(x,y) = (x + 2y - 7)^2 + (2x + y - 5)^2$$

Старт: $(-1, -1)$

6. **Матьяса**

$$f(x,y) = 0.26(x^2 + y^2) - 0.48xy$$

Старт: $(3,3)$

7. **Леви**

$$f(x,y) = \sin^2(3\pi x) + (x-1)^2\big(1 + \sin^2(3\pi y)\big) + (y-1)^2\big(1 + \sin^2(2\pi y)\big)$$

Старт: $(2,2)$

8. **Трёхгорбого верблюда**

$$f(x,y) = 2x^2 - 1.05x^4 + \frac{x^6}{6} + xy + y^2$$

Старт: $(-2, -2)$

9. **Экли**

$$f(x,y) = -20\exp\left(-0.2\sqrt{0.5(x^2+y^2)}\right) - \exp\left(0.5\big(\cos(2\pi x) + \cos(2\pi y)\big)\right) + e + 20$$

Старт: $(2,2)$

10. **Седловая квадратичная**

$$f(x,y) = x^2 - y^2 + 0.1(x^4 + y^4)$$

Старт: $(0.2, 0.2)$

5.2. Таблица вариантов

Таблица 2.3. Варианты индивидуальных заданий

№	Функция	Набор данных	Модель	Дополнительно
1	Розенброк	MNIST	MLP	Сравнение скорости сходимости
2	Химмельблау	FashionMNIST	CNN	Анализ локальных минимумов
3	Била	California Housing	MLP	Регрессия, сравнение оптимизаторов
4	Растригин	CIFAR-10, подвыборка	CNN	Влияние шума градиента
5	Бута	EMNIST	MLP	Устойчивость к выбору lr
6	Матьясова	KMNIST	CNN	Быстрая сходимость и переобучение
7	Леви	CIFAR-100, подвыборка	CNN	Сравнение качества обобщения
8	Трёхгорбого верблюда	USPS	MLP	Поведение вблизи седловых точек
9	Экли	SVHN, подвыборка	CNN	Исследование осцилляций траекторий
10	Седловая квадратичная	MNIST	MLP	Сравнение SGD и Adam в седловой области

6. Вопросы для отчёта и защиты

1. Сформулируйте задачу обучения нейронной сети как задачу минимизации эмпирического риска.
2. Почему полный градиент по всей выборке часто заменяют мини-пакетным градиентом?
3. Как размер мини-пакета влияет на дисперсию градиентной оценки?
4. Какой эффект даёт использование инерции в SGD?
5. Почему Adam обычно сходится быстрее на начальных этапах обучения?
6. Зачем в Adam используется поправка на смещение оценок моментов?
7. В чём состоит различие между L_2-регуляризацией и weight decay в адаптивных оптимизаторах?
8. Почему слишком большая скорость обучения может привести к неустойчивому обучению?
9. Почему слишком маленький мини-пакет иногда ухудшает стабильность обучения?
10. Какой из изученных оптимизаторов в вашем эксперименте дал лучшую обобщающую способность и почему?

7. Требования к содержанию отчёта

Отчёт должен включать:

1. Титульную информацию.
2. Цель и задачи работы.
3. Краткую теоретическую справку.
4. Листинг реализации пользовательских оптимизаторов.
5. Результаты этапа с 2D-функцией:

– траектории;

– конечные значения функции;
– выводы.

6. Результаты обучения нейронной сети:
– графики потерь;
– графики точности;
– итоговую таблицу метрик.
7. Результаты исследования гиперпараметров.
8. Общие выводы по работе.
9. Ответы на контрольные вопросы.

Литература

Основная литература

1. Николенко С. И., Кадурин А., Архангельская Е. Глубокое обучение. — СПб.: Питер, 2018. — 416 с.
2. Бишоп К. М., Бишоп Х. Глубокое обучение: основы и концепции. — М.: ДМК Пресс, 2024. — 742 с.
3. Раска С., Мирджалили В. Машинное обучение на PyTorch и Scikit-Learn. — М.: ДМК Пресс, 2023. — 580 с.
4. Чолле Ф. Глубокое обучение на Python / пер. с англ. — 2-е изд. — СПб.: Питер, 2022. — 576 с.
5. Жерон О. Прикладное машинное обучение с помощью Scikit-Learn, Keras и TensorFlow / пер. с англ. — 2-е изд. — СПб.: Питер, 2021. — 736 с.

Ключевые публикации по теме лабораторной работы

6. Роббинс Г., Монро С. Стохастический метод аппроксимации // Анналы математической статистики. — 1951. — Т. 22, № 3. — С. 400–407.
7. Поляк Б. Т. Некоторые методы ускорения сходимости итеративных процессов // Журнал вычислительной математики и математической физики. — 1964. — Т. 4, № 5. — С. 799–809.
8. Суцкевер И., Мартенс Дж., Даль Дж., Хинтон Г. О важности инициализации и импульса в глубоком обучении // Труды 30-й Международной конференции по машинному обучению (ICML’13). — 2013. — Т. 28, № 3. — С. 1139–1147.
9. Кингма Д. П., Ба Дж. Адам: метод стохастической оптимизации // Архив научных публикаций. — 2014.— Режим доступа: https://arxiv.org/abs/1412.6980 (дата обращения: 15.06.2026).
10. Лощилов И., Хаттер Ф. Раздельнный весовой распад для регуляризации // Архив научных публикаций. — 2017. — Режим доступа: https://arxiv.org/abs/1711.05101 (дата обращения: 15.06.2026).
11. Ботту Л., Кёртис Ф. Э., Носедал Дж. Методы оптимизации для крупномасштабного машинного обучения // Обзор СИМА. — 2018. — Т. 60, № 2. — С. 223–311.
12. Рудер С. Обзор алгоритмов градиентного спуска для оптимизации // Архив научных публикаций. — 2016. — Режим доступа: https://arxiv.org/abs/1609.04747 (дата обращения: 15.06.2026).
13. Уилсон А. С., Роэлофс Р., Стерн М., Сребро Н., Рехт Б. Предельная ценность адаптивных градиентных методов в машинном обучении // Достижения в системах обработки нейронной информации (NeurIPS 2017). — 2017. — Т. 30. — С. 4148–4158.

Официальная документация и онлайн-ресурсы

14. Оптимизаторы в PyTorch [Электронный ресурс]. — Режим доступа: https://pytorch.org/docs/stable/optim.html (дата обращения: 15.06.2026).

15. Воспроизводимость экспериментов в PyTorch [Электронный ресурс]. — Режим доступа: https://pytorch.org/docs/stable/notes/randomness.html (дата обращения: 15.06.2026).
16. Оптимизаторы в TensorFlow Keras [Электронный ресурс]. — Режим доступа: https://www.tensorflow.org/api_docs/python/tf/keras/optimizers (дата обращения: 15.06.2026).
17. Документация по TensorBoard [Электронный ресурс]. — Режим доступа: https://www.tensorflow.org/tensorboard (дата обращения: 15.06.2026).
18. Чжан А., Липтон З. К., Ли М., Смола А. Дж. Погружение в глубокое обучение [Электронный ресурс]. — Режим доступа: https://d2l.ai (дата обращения: 15.06.2026).

Дополнительные источники для углублённого изучения

19. Гудфеллоу И., Бенджио И., Курвиль А. Глубокое обучение. — М.: ДМК Пресс, 2018. — 800 с.
20. Шмидхубер Ю. Глубокое обучение в нейронных сетях: обзор // Нейронные сети. — 2015. — Т. 61. — С. 85–117.
21. Смит Л. Н. Циклические скорости обучения для обучения нейронных сетей // Труды Зимней конференции по применению компьютерного зрения (WACV 2017). — 2017. — С. 461–470.
22. Ху Б., Хемант К., Чен Ю. Теоретический анализ адаптивного градиентного метода Адам // Труды 4-й Международной конференции по представлениям обучения (ICLR 2020). — 2020. — Режим доступа: https://arxiv.org/abs/1910.05446 (дата обращения: 15.06.2026).